

Documentación de appian-flow

La librería **appian-flow** es un habilitador de automatización para aplicaciones Appian, diseñada para proporcionar a los usuarios un conjunto completo de funcionalidades que permiten interactuar con flujos de trabajo de manera automatizada mediante Selenium.

Este framework permite ejecutar procesos de negocio sobre Appian como:

- Búsqueda de casos
- Consulta de información
- Avance de casos en flujo operativo
- Creación de nuevos casos
- Descarga de reportes desde Process HQ
- Descarga de adjuntos

El **objetivo principal** es abstraer la complejidad de la interfaz web y exponer una librería clara, reutilizable y mantenible para automatización de procesos.

Contenido del paquete

Los siguientes archivos se encuentran en **src/an0016001_appian_flow**

Nombre del archivo	Descripción
---	---
`appian_flow.py`	Clase principal orquestadora del flujo de automatización
`driver_manager.py`	Configuración y creación del WebDriver
`app_state.py`	Control del estado de la aplicación y validación de carga
`login_page.py`	Manejo del proceso de autenticación
`cases_page.py`	Gestión completa de casos (búsqueda, consulta, avance y creación)
`report_page.py`	Descarga de reportes desde Process HQ
`form_handler.py`	Manejo dinámico de formularios y tipos de campos
`download_manager.py`	Control y validación de descargas de archivos
`xpath_builder.py`	Generación centralizada de selectores XPath
`response.py`	Estandarización de respuestas del sistema
`__init__.py`	Inicialización del paquete

Instalación de la librería

Los pasos para instalar appian-flow como librería son:

- Crear un ambiente virtual sobre el cual se va a ejecutar, empleando el comando

```
python -m venv venv
```

- Activar el ambiente virtual y actualizar la última versión del paquete pip empleando los comandos

```
.\venv\Scripts\activate
```

- Actualizar el gestor de paquetes pip

```
python -m pip install -U pip
```

- Instalar la librería de appian-flow empleando el comando

```
pip install an0016001_appian_flow
```

- Para actualizar la librería de appian-flow empleando el comando

```
pip install --upgrade an0016001_appian_flow
```

Uso de la librería

Instanciar librería

Inicializar los parámetros de configuración para la automatización.
Para utilizar la librería, primero se debe crear una instancia de `AppianFlow`:

```
python
from an0016001_appian_flow.appian_flow import AppianFlow

flow = AppianFlow(
    url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
    download_dir=r"C:\Users\xxxxx\Downloads",
    browser="edge",
    timeout=60,
    timeout_files=600
)
```

Parámetros

Parámetro	Tipo	Descripción
-----	-----	-----
url	str	URL de la aplicación
download_dir	str (opcional)	Ruta donde se guardarán las descargas
browser	str (opcional)	Navegador a utilizar (por defecto "edge")
timeout	int (opcional)	Tiempo de espera para carga de elementos
timeout_files	int (opcional)	Tiempo máximo de espera para descargas

Instanciar Métodos

Después de crear una instancia de `AppianFlow`, se pueden llamar los siguientes métodos:

1. Método: start()

Inicia la aplicación y realiza el proceso de autenticación si es requerido.
Si el login no es requerido, no se debe enviar usuario ni contraseña.

Detalles:

- Inicializa el navegador y abre la URL de la aplicación
- Verifica si se requiere autenticación.
- Ejecuta autenticación si aplica.
- Despues de iniciar la aplicación, se pueden llamar los métodos de negocio (buscar caso, avanzar caso, crear caso, descargar reporte).

Parámetros

Parámetro	Tipo	Descripción
-----	-----	-----
user	str (opcional)	Usuario de acceso
password	str (opcional)	Contraseña

Notas:

Si el entorno requiere login, user y password son obligatorios.

Si falla el proceso, retorna error controlado.

Retorno

```
"""json
{
  "success": true,
  "message": "Aplicación iniciada correctamente",
  "data": null
}
"""
```

Ejemplo de start() y como se debe instanciar para su ejecución dentro del código tal y como lo muestra el siguiente ejemplo:

-- Si el ambiente no requiere usuario y contraseña

```
"""python
from an0016001_appian_flow.appian_flow import AppianFlow

flow = AppianFlow(
    url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
    download_dir=r"C:\Users\xxxxx\Downloads"
)

flow.start()
"""
```

-- Si el ambiente requiere usuario y contraseña

```
"""python
from an0016001_appian_flow.appian_flow import AppianFlow

flow = AppianFlow(
```

```

        url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
        download_dir=r"C:\Users\xxxxx\Downloads"
    )

    flow.start(
        user=xxxx,
        password=xxxx
    )
'''

```

Posibles errores

- Error de conexión con la URL
- Credenciales inválidas
- Timeout en la carga de la aplicación

2. Método: search_case(case_id)

Busca un caso en la aplicación y lo abre si existe.

Detalles:

- Accede al módulo de Seguimiento de solicitudes
- Ingresa el `case\_id` en el buscador
- Ejecuta la búsqueda
- Valida si el caso existe
- Abre el caso si es encontrado

Parámetros

Parámetro	Tipo	Descripción
-----	-----	-----
case_id	str	Identificador del caso

Retorno

```

'''json
{
    "success": true,
    "message": "OK",
    "data": null
}
'''

```

Ejemplo de search_case() y como se debe instanciar para su ejecución dentro del código tal y como lo muestra el siguiente ejemplo:

```

'''python
from an0016001_appian_flow.appian_flow import AppianFlow

```

```

flow = AppianFlow(
    url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
    download_dir=r"C:\Users\xxxxx\Downloads"
)

flow.start()

response = flow.search_case(case_id)
'''

```

Posibles errores

- Caso no encontrado
- Error de carga en la bandeja de casos
- Timeout en búsqueda

3. Método: `get_case_data(case_id, download_attachments=False)`

Obtiene la información detallada de un caso.

Extrae todos los campos visibles organizados en secciones y los retorna como un DataFrame.

Detalles:

- Busca el caso
- Accede al detalle del caso
- Extrae las secciones visibles
- Construye un DataFrame con label y value
- Descarga adjuntos si `download_attachments = True`

Nota:

Siempre antes de llamar al método `get_case_data()` se debe llamar al método `search_case()`, para que busque el caso y lo abra.

Parámetros

Parámetro	Tipo	Descripción
<code>case_id</code>	str	Identificador del caso
<code>download_attachments</code>	bool	Indica si se deben descargar los adjuntos del caso.

Retorno

Campos dentro de `'data'`:

- `'info'`: DataFrame con columnas:
 - `'label'`
 - `'value'`
- `'files'`: lista de archivos descargados (si aplica)

```

"""json
{
  "success": true,
  "message": "Consulta de caso ejecutada correctamente",
  "data": {
    "info": "DataFrame",
    "files": []
  }
}
"""

```

Ejemplo de `get_case_data()` y como se debe instanciar para su ejecución dentro del código tal y como lo muestra el siguiente ejemplo:

```

"""python
from an0016001_appian_flow.appian_flow import AppianFlow

flow = AppianFlow(
    url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
    download_dir=r"C:\Users\xxxxx\Downloads"
)

flow.start()

response = flow.search_case(case_id)

if response["success"] == False:
    print(response["message"])
    flow.cerrar()

response = flow.get_case_data(case_id, download_attachments=False)
"""

```

Posibles errores

- Caso no encontrado
- Error al leer los campos del formulario
- Error en descarga de adjuntos

4. Método: `advance_case(case_id, form_data)`

Permite avanzar un caso mediante el diligenciamiento automático de un formulario dinámico asociado al flujo.

Detalle:

- Validación de existencia del caso
- Validación de asignación del caso
- Toma automática de la tarea (si aplica)
- Verificación de concurrencia (bloqueo si ya fue tomado)
- Construcción dinámica del formulario basado en labels

- Envío del formulario
- Finalización de la tarea/proceso

Nota:

Siempre antes de llamar al método `advance\_case()` se debe llamar al método `search\_case()`, para que busque el caso y lo abra.

Parámetros

Parámetro	Tipo	Descripción
case_id	str	Identificador del caso
form_data	dict	Datos del formulario dinámico

form_data (dict): Datos para completar el formulario.

- nombre_del_campo: Nombre del campo en el formulario
- valor: Valor a ingresar o seleccionar en el campo
- Ejemplo:

```
form_data = {
    "Fecha Registro": "04/05/2026",
    "Cómo Deseas Atender Esta Solicitud": "Complementar Información (Devolver)",
    "Seleccione la Actividad": "Realizar Solicitud",
    "Desea Enviar Notificación Personalizada": "Si",
    "Seleccione el Usuario o los Usuarios": ['abechav'],
    "Comentario a Enviar en la Notificación": "Se requiere complementar información",
    "Observaciones": "Se requiere complementar la información",
    "Adjuntos": [
        r"C:\Users\abechav\Downloads\id_eucs.xlsx"
    ]
}
```

Reglas para `form\_data`

- Las llaves deben coincidir exactamente con el **label del campo** en el formulario
- Los valores deben corresponder a:
 - Texto libre en campos tipo input
 - Opciones válidas en campos tipo select/radio
- El matching es **case-sensitive**
- Si un campo no es encontrado, generará error

Retorno

```
"""json
{
    "success": true,
    "message": "OK",
    "data": null
}
"""
```

Ejemplo de advance_case() y como se debe instanciar para su ejecución dentro del código tal y como lo muestra el siguiente ejemplo:

```

"""python
from an0016001_appian_flow.appian_flow import AppianFlow

flow = AppianFlow(
    url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
    download_dir=r"C:\Users\xxxxx\Downloads"
)

flow.start()

response = flow.search_case(case_id)

if response["success"] == False:
    print(response["message"])
    flow.cerrar()

form_data = {
    "Cómo Deseas Atender Esta Solicitud": "Finalizar",
    "Cómo Deseas Finalizar Esta Solicitud": "Finalizado No Exitoso",
    "Desea Enviar Notificación Personalizada": "No",
    "Observaciones": "Ok pruebas",
    "Adjuntos": [
        r"C:\Users\labechav\Downloads\lid_eucs.xlsx"
    ]
}
response = flow.advance_case(case_id, form_data)
"""

```

Posibles errores

- Caso no existe
- Caso ya asignado a otro usuario
- Error en el diligenciamiento del formulario
- Campos inválidos en `form\_data`

Ejemplo de respuesta de error:

```

"""json
{
    "success": false,
    "message": "El caso ya fue tomado por otro usuario",
    "data": null
}
"""

```


5. Método: `create_case(category_name, flow_name, form_data)`

Crea un nuevo caso en Appian.

Detalles:

- Accede al módulo de Radicación de solicitudes
- Selecciona la categoría (`category_name`)
- Selecciona el flujo (`flow_name`)
- Construye el formulario dinámicamente (`form_data`)
- Diligencia los campos
- Adjunta archivos (si aplica)
- Envía el formulario
- Captura el número de caso generado

Parámetros

Parámetro	Tipo	Descripción
<code>category_name</code>	str	Nombre de la categoría
<code>flow_name</code>	str	Nombre del flujo
<code>form_data</code>	dict	Datos del formulario

`form_data` (dict): Datos del formulario.

- `nombre_del_campo`: Nombre del campo en el formulario
- `valor`: Valor a ingresar o seleccionar en el campo
- **Ejemplo:**

```
form_data = {
    "Fecha Registro": "04/05/2026",
    "Cuenta Contable": "111111111111",
    "Valor del Registro": 2000.34,
    "Observaciones": "Radicación automática",
    "Asignar a": ["abechav"],
    "Adjuntos": [
        r"C:\\Users\\abechav\\Downloads\\FCO_EUC_06_04.xlsx"
    ]
}
```

Reglas para `form_data`

- Las llaves deben coincidir con el label del campo
- Si un campo no es encontrado, generará error
- Tipos soportados:
 - `str`: campos de texto
 - `list`: multiselect o usuarios
 - `list[str]`: rutas de archivos para adjuntos
- El campo de adjuntos debe contener rutas válidas

Retorno

Detalles:

Retorna el número de solicitud generado.

```
```json
{
 "success": true,
 "message": "OK",
 "data": "numero_caso"
}
```
```

Ejemplo de create_case() y como se debe instanciar para su ejecución dentro del código tal y como lo muestra el siguiente ejemplo:

```
```python
from an0016001_appian_flow.appian_flow import AppianFlow

flow = AppianFlow(
 url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
 download_dir=r"C:\Users\xxxxx\Downloads"
)

flow.start()

category_name = "Sufi"
flow_name = "Gestiones Contables Sufi"
form_data={
 "Fecha Registro": "04/05/2026",
 "Cuenta Contable": "1111111111111",
 "NIT del Registro": "1111111111111",
 "Observaciones": "Radicación automática",
 "Asignar a": ["abechav"],
 "Adjuntos": [
 r"C:\Users\abechav\Downloads\id_eucs.xlsx",
 r"C:\Users\abechav\Downloads\FCO_EUC_06_04.xlsx",
],
 "Desea Enviar Notificación Personalizada": "No"
}

response = flow.create_case(
 category_name,
 flow_name,
 form_data
)
```
```

Posibles errores

- Categoría no encontrada
- Flujo no disponible
- Error en diligenciamiento de campos
- Archivos no encontrados
- Error al crear el caso

6. Método: `download_report(report_data, filter_data)`

Descarga un reporte desde Process HQ aplicando filtros dinámicos.

Detalles:

- Accede al reporte en Process HQ
- Aplica filtros dinámicos
- Ejecuta la búsqueda
- Genera el archivo
- Espera la descarga
- Valida que el archivo exista

Parámetros

| Parámetro | Tipo | Descripción |
|--------------------------|------|---------------------|
| <code>report_data</code> | dict | Datos del reporte |
| <code>filter_data</code> | dict | Filtros del reporte |

`report_data` (dict): Datos del reporte.

- `flow_name`: Nombre del flujo asociado al reporte
- `report_name`: Nombre del reporte a descargar
- **Ejemplo:**

```
report_data = {  
    "flow_name": "Solicitud investigación de bienes",  
    "report_name": "Informe detallado Solicitud investigación de bienes"  
}
```

`filter_data` (dict): Filtros a aplicar en el reporte.

- `nombre_del_campo`: Nombre del campo a filtrar
- `valor`: Valor para el filtro
- **Ejemplo:**

```
filter_data={  
    "Solicitud fecha de creación": ["01/03/2026", "29/04/2026"],  
    "Estado de la solicitud": "Finalizado exitoso"  
}
```

Reglas para ``report_data`` y ``filter_data``

- Las llaves deben coincidir con los labels de los filtros
- Tipos soportados:
 - ``str``: filtros simples
 - ``list[str]``: rangos de fechas o selección múltiple

Retorno

```
```json
{
 "success": true,
 "message": "OK",
 "data": {
 "file": {},
 "path": {},
 "full_path": {}
 }
}
```

```python
from an0016001_appian_flow.appian_flow import AppianFlow

flow = AppianFlow(
 url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
 download_dir=r"C:\Users\xxxxx\Downloads"
)

flow.start()

report_data = {
 "flow_name": "Solicitud investigación de bienes",
 "report_name": "Informe detallado Solicitud investigación de bienes"
}

filter_data={
 "Solicitud fecha de creación": ["01/03/2026", "29/04/2026"],
 "Estado de la solicitud": "Finalizado exitoso"
}

response = flow.download_report(report_data, filter_data)
```
```

Posibles errores

- Error al cargar el reporte
- Filtros inválidos
- Timeout en descarga
- Archivo no generado

Estructura de respuestas

Todos los métodos retornan un objeto estándar:

```
"""json
{
  "success": bool,
  "message": str,
  "data": any
}
"""
```

Prerrequisitos

- Python 3.10+
- Navegador Edge o Chrome instalado
- WebDriver compatible
- Acceso a Appian

Dependencias:

- 'selenium==4.36.0',
- 'pandas>=1.5.0'

Manejo de sesión

Se recomienda reutilizar la instancia de `AppianFlow` para múltiples operaciones, evitando reinicializar el navegador repetidamente.

Buenas prácticas

- No hardcodear credenciales
- Validar los labels antes de construir `form\_data`
- Manejar respuestas mediante `success`
- Usar timeouts adecuados

Ejercicio práctico

```
"""python
from an0016001_appian_flow.appian_flow import AppianFlow

if __name__ == "__main__":

    flow = AppianFlow(
        url="https://smartprocess-qa.bancolombia.com/suite/sites/icc",
        download_dir=r"C:\Users\xxxxxx\Downloads"
    )

    response = flow.start()
    if response["success"] == False:
        print("Error en start:", response["message"])
```

```

flow.cerrar()

# Descargar Informes
report_data = {
    "flow_name": "Solicitud investigación de bienes",
    "report_name": "Informe detallado Solicitud investigación de bienes"
}

filter_data={
    "Solicitud fecha de creación": ["01/03/2026", "29/04/2026"]
}

response = flow.download_report(report_data, filter_data)

if response["success"]:
    data = response["data"]
    print("Archivo:", data["file"])
    print("Ruta:", data["path"])
    print("Ruta completa:", data["full_path"])
else:
    print("Error:", response["message"])

# Consultar caso
case_id = "GCS-31360"
response = flow.search_case(case_id)

if response["success"] == False:
    print(response["message"])
    continue
    flow.cerrar()

# Obtener la información del caso
response = flow.get_case_data(case_id, False)

if response["success"] == False:
    print(response["message"])
    continue
    flow.cerrar()
else:
    #datos del caso
    for _, row in response["data"]["info"].iterrows():
        print(f"{row['label']}: {row['value']}")

    #los archivos adjuntos
    for file in response["data"]["files"]:
        print("Archivo descargado:", file['full_path'])

# Avanzar Caso
form_data = {
    "Cómo Deseas Atender Esta Solicitud": "Finalizar",
    "Cómo Deseas Finalizar Esta Solicitud": "Finalizado No Exitoso",

```

```

        "Desea Enviar Notificación Personalizada": "No",
        "Observaciones": "Ok pruebas"
    }

    response = flow.advance_case(case_id, form_data)
    if response["success"] == False:
        print("Error en avanzar caso:", response["message"])

# Radicar Caso
category_name = "Sufi"
flow_name = "Gestiones Contables Sufi"
form_data={
    "Fecha Registro": "04/05/2026",
    "Cuenta Contable": "111111111111",
    "NIT del Registro": "111111111111",
    "Número de Referencia": "222222",
    "Valor del Registro": 2000.34,
    "Área": "TI",
    "Tipo de Solicitud": "Gestionar inconsistencia",
    "Tipología de la Inconsistencia": "Impuestos",
    "¿La Inconsistencia se Originó por la Operación de Desembolsos?": "No",
    "¿Identifica si es Definición Proceso o Error Usuario?": "Error usuario",
    "Observaciones": "Radicación automática",
    "Asignar a": ["abechav"],
    "Adjuntos": [
        r"C:\Users\xxxxx\Downloads\lid_eucs.xlsx",
        r"C:\Users\xxxxx\Downloads\FCO_EUC_06_04.xlsx",
    ],
    "Desea Enviar Notificación Personalizada": "No"
}

response = flow.create_case(
    category_name,
    flow_name,
    form_data
)
if response["success"] == False:
    print("Error en radicar solicitud:", response["message"])
    flow.cerrar()

print("Número de solicitud radicada:", response["data"])
'''

```